

FlexQL Design Document

GitHub Repository: <https://github.com/ritviksahu/FlexQL>

1 Overview

FlexQL is a custom SQL server implemented fully in C++. The system exposes:

- a TCP server,
- a C-compatible client API,
- an interactive REPL client built on top of that API,
- benchmark tools for inserts, query workloads, and stress testing.

The implementation does not use SQLite or any other external database library. The full query parsing, execution, persistence, recovery, and storage logic is implemented inside the project codebase.

The system is designed around the project constraints:

- Persistent disk-backed storage is the primary source of truth.
- RAM is used only to accelerate query execution and insert throughput.
- The server is fault-tolerant through a journal and insert write-ahead log (WAL).
- Batch insertion is supported and optimized.
- The required SQL subset, including comparison operators in `WHERE` and `JOIN`, is supported.

2 Repository Contents

Important files in the repository are:

- `src/flexql_server.cpp`: custom server, parser, execution engine, storage engine, WAL, and recovery logic.
- `src/flexql.cpp`: client-side API implementation.
- `src/client.cpp`: interactive REPL client using the FlexQL API.
- `flexql.h`: public C-compatible API used by the client and benchmarks.
- `benchmark_flexql.cpp`: provided benchmark file integrated into the codebase.
- `benchmark_select.cpp`: focused query benchmark for point, range, join, and count queries.
- `stress_benchmark.cpp`: extra read-heavy, write-heavy, and mixed workload benchmark.
- `compile.sh`: build script.

3 Data Storage Design

3.1 Persistent Layout

All persistent data is stored under the `dbdata/` directory. Disk is the primary storage layer.

For each table, the engine stores:

- a metadata file (`.meta`) containing the table schema,
- a table data file (`.data`) containing persisted rows,
- a global insert WAL (`insert.wal`) for durable batched inserts,
- a structural journal (`journal.txt`) used for `CREATE TABLE` and `DROP TABLE`.

3.2 Row Representation

Rows are stored in a compact binary format in the persistent data files and WAL. Each row contains:

- a row identifier,
- the number of column values,
- the serialized values for each column.

This binary layout avoids repeated reparsing during recovery and checkpointing.

3.3 Durability and Recovery

The engine uses two recovery mechanisms:

- **Structural journal:** used for schema-level operations such as table creation and deletion.
- **Insert WAL:** all acknowledged inserts are written durably to disk before the server replies to the client.

On startup, the server:

1. loads table metadata and persisted rows from disk,
2. replays any unfinished journal operation,
3. replays any committed-but-not-checkpointed rows from the insert WAL,
4. checkpoints replayed data back to table files.

This design ensures that crashes do not lose acknowledged inserts and that structural changes can be safely recovered.

4 Indexing Method

4.1 Equality Indexes

The server maintains lazy in-memory equality indexes for columns. These indexes map a column value to the set of matching row positions. They are primarily used for:

- point lookups such as `WHERE ID = 123`,
- equality joins such as `ON A.ID = B.USER_ID`.

The equality indexes are built lazily, so insert-heavy workloads do not pay index maintenance cost until a query actually needs a specific column index.

4.2 Numeric Range Indexes

The server also maintains lazy numeric range indexes for numeric columns. Each such index stores pairs of:

- numeric column value,
- row position.

The vector is kept sorted so numeric range predicates can use binary search rather than scanning the full table. These indexes support:

- $>$,
- $<$,
- $>=$,
- $<=$.

This improves range filters and filtered `COUNT(*)` queries.

5 Caching Strategy

RAM is not the primary store. Instead, memory is used as a secondary acceleration layer:

- rows are cached in memory after loading,
- equality indexes are built lazily on demand,
- numeric range indexes are built lazily on demand,
- prepared insert batches and checkpoint buffers use temporary memory to reduce system call overhead.

The authoritative durable copy remains on disk in `dbdata/`. If the process crashes, the server reconstructs its in-memory state from persistent files plus WAL replay.

6 Handling of Expiration Timestamps

The benchmark data includes expiration timestamps such as `EXPIRES_AT`. In the current implementation:

- expiration timestamps are stored as normal numeric column values,
- they can be filtered through standard SQL conditions,
- they are persisted and recovered like any other column.

The server does not currently run a background garbage collector that automatically deletes expired rows. This was a deliberate design choice because the project requirements and benchmark treat expiration timestamps as queryable data rather than automatic deletion triggers.

7 Query Handling and Execution

7.1 Supported SQL Subset

The custom parser and executor support:

- CREATE TABLE,
- DROP TABLE IF EXISTS,
- INSERT INTO ... VALUES (...), (...), ...,
- SELECT,
- SELECT COUNT(*),
- INNER JOIN,
- WHERE conditions using =, >, <, >=, and <=.

7.2 Parsing

The engine uses:

- a specialized fast parser for hot-path batched INSERT statements,
- a token-based parser for CREATE, DROP, and SELECT.

This split reduces insert overhead while keeping the implementation manageable.

7.3 Execution Strategy

- Point predicates prefer equality indexes.
- Range predicates prefer the numeric range index when available.
- Joins use equality indexes when the join predicate is equality-based.
- If no index applies, the engine falls back to scanning candidate rows.

8 Batch Insertion Strategy

Batch insertion is a central optimization in the design. The server accepts multi-row statements of the form:

```
INSERT INTO T VALUES (...), (...), (...);
```

This reduces:

- network round-trip overhead,
- parser setup overhead,
- per-statement WAL overhead.

The benchmark parameter `INSERT_BATCH_SIZE` was tuned experimentally, and a batch size of 1000 gave strong performance on the target environment.

9 Multithreading Design

The server uses a dedicated-thread-per-client model:

- the TCP server listens on `127.0.0.1:9000`,
- each accepted client connection is handled by its own thread,
- shared database state is protected with a shared mutex.

This design is simple and practical for the project scope. A production system might instead use a thread pool or asynchronous event loop, but the current design is easier to reason about and sufficient for concurrent benchmark and client access.

10 Interactive Terminal Interface (REPL)

The project includes an interactive terminal client implemented in `src/client.cpp`. This REPL uses the public FlexQL API from `flexql.h`, satisfying the requirement that the client terminal interface be built on the provided API layer.

The REPL:

- connects to the running FlexQL server,
- accepts SQL statements from the terminal,
- executes them through `flexql_exec`,
- prints query results and errors,
- continues until the user enters `quit` or `exit`.

11 Other Design Decisions

11.1 Large Checkpoint Threshold

The engine uses batched checkpointing. Inserts are first persisted durably to the WAL, and only later checkpointed into table data files. The checkpoint threshold is intentionally large so that large bulk loads spend most of their time on sequential WAL appends instead of repeatedly rewriting table files.

11.2 Open File Handles

The insert WAL and table data files are kept open across requests. This reduces repeated open/close overhead in the hot path.

11.3 Move-Based Insert Path

Insert rows are moved through the execution path where possible instead of copied. This reduces allocation and string-copy overhead during batched inserts.

11.4 Release Build Configuration

The project is built in release mode using:

- `-O3`
- `-DNDEBUG`

This improves parser, serialization, scan, and comparison performance.

12 Compilation and Execution Instructions

12.1 Compilation

From the repository root:

```
sh compile.sh
```

12.2 Clean Start

To rerun from a clean state:

```
lsof -ti tcp:9000 | xargs kill 2>/dev/null
rm -rf dbdata
```

12.3 Start the Server

```
./build/server
```

12.4 Run the Interactive REPL Client

```
./build/client
```

Custom host and port:

```
./build/client 127.0.0.1 9000
```

12.5 Run the Provided Benchmark

```
./build/benchmark
```

Run only unit tests:

```
./build/benchmark --unit-test
```

Run a large insert benchmark:

```
./build/benchmark 10000000
```

12.6 Run the Extra Stress Benchmark

```
./build/stress_benchmark
```

Custom:

```
./build/stress_benchmark 5000 4 1000 25
```

12.7 Run the Query Benchmark

```
./build/benchmark_select
```

Custom:

```
./build/benchmark_select 100000 1000 5 1000
```

13 Performance Results for Large Datasets

The project was evaluated with the provided benchmark as well as additional stress and query benchmarks.

13.1 Large Insert Benchmark

The system successfully inserted 10,000,000 rows while preserving persistence and passing the unit tests. A representative result from the optimized implementation was:

- inserted rows: 10,000,000
- elapsed time: approximately 64.7 seconds
- throughput: approximately 154,459 rows/sec
- correctness: 21/21 unit tests passed

13.2 Observed Query Performance

The extra query benchmark showed the expected behavior:

- point lookups are fast because they use equality indexes,
- equality joins are also fast,
- range and count queries are heavier because they process larger candidate sets,
- numeric range indexing improves planner support for numeric comparisons.

14 Benchmark Result Screenshots

```
ritvik@Ritviks-MacBook-Air FLEXQL % ./build/benchmark 10000000
Connected to FlexQL
Running SQL subset checks plus insertion benchmark...
Target insert rows: 10000000

[PASS] CREATE TABLE BIG_USERS (1 ms)

Starting insertion benchmark for 10000000 rows...
Progress: 1000000/10000000
Progress: 2000000/10000000
Progress: 3000000/10000000
Progress: 4000000/10000000
Progress: 5000000/10000000
Progress: 6000000/10000000
Progress: 7000000/10000000
Progress: 8000000/10000000
Progress: 9000000/10000000
Progress: 10000000/10000000
[PASS] INSERT benchmark complete
Rows inserted: 10000000
Elapsed: 9126 ms
Throughput: 1095770 rows/sec

[[...Running Unit Tests...]]

[PASS] CREATE TABLE TEST_USERS (0 ms)
[PASS] INSERT TEST_USERS ID=1 (0 ms)
[PASS] INSERT TEST_USERS ID=2 (0 ms)
[PASS] INSERT TEST_USERS ID=3 (0 ms)
[PASS] INSERT TEST_USERS ID=4 (0 ms)
[PASS] Basic SELECT * validation
[PASS] Single-row value validation
[PASS] Filtered rows validation
[PASS] Empty result-set validation
[PASS] CREATE TABLE TEST_ORDERS (0 ms)
[PASS] INSERT TEST_ORDERS ORDER_ID=101 (0 ms)
[PASS] INSERT TEST_ORDERS ORDER_ID=102 (0 ms)
[PASS] INSERT TEST_ORDERS ORDER_ID=103 (0 ms)
[PASS] Join with no matches validation
[PASS] Invalid SQL should fail
[PASS] Missing table should fail

Unit Test Summary: 21/21 passed, 0 failed.
```

Figure 1: Benchmark result screenshot ss1.png.


```
ritvik@Ritviks-MacBook-Air FLexQL % ./build/benchmark --unit-test
Connected to FLexQL

[[...Running Unit Tests...]]

[PASS] CREATE TABLE TEST_USERS (0 ms)
[PASS] INSERT TEST_USERS ID=1 (0 ms)
[PASS] INSERT TEST_USERS ID=2 (0 ms)
[PASS] INSERT TEST_USERS ID=3 (0 ms)
[PASS] INSERT TEST_USERS ID=4 (0 ms)
[PASS] Basic SELECT * validation
[PASS] Single-row value validation
[PASS] Filtered rows validation
[PASS] Empty result-set validation
[PASS] CREATE TABLE TEST_ORDERS (0 ms)
[PASS] INSERT TEST_ORDERS ORDER_ID=101 (0 ms)
[PASS] INSERT TEST_ORDERS ORDER_ID=102 (0 ms)
[PASS] INSERT TEST_ORDERS ORDER_ID=103 (0 ms)
[PASS] Join with no matches validation
[PASS] Invalid SQL should fail
[PASS] Missing table should fail

Unit Test Summary: 21/21 passed, 0 failed.
```

Figure 2: Benchmark result screenshot ss2.png.

```
ritvik@Ritviks-MacBook-Air FLexQL % ./build/stress_benchmark
Preparing workload tables...
Seed rows: 5000
Threads: 4
Operations per thread: 1000
Write batch size: 25

[Read-heavy workload]
Elapsed: 32 ms
Successful ops: 4000
Failed ops: 0
Total ops: 4000
Throughput: 125000 ops/sec
Average successful op latency: 30 us

[Write-heavy workload]
Elapsed: 5 ms
Successful ops: 4000
Failed ops: 0
Total ops: 4000
Throughput: 800000 ops/sec
Average successful op latency: 5 us

[Mixed multi-client workload]
Elapsed: 56 ms
Successful ops: 4000
Failed ops: 0
Total ops: 4000
Throughput: 71428 ops/sec
Average successful op latency: 54 us

Overall result: PASS
```

Figure 3: Benchmark result screenshot ss3.png.

```
ritvik@Ritviks-MacBook-Air FLexQL % ./build/stress_benchmark 5000 4 1000 25
Preparing workload tables...
Seed rows: 5000
Threads: 4
Operations per thread: 1000
Write batch size: 25

[Read-heavy workload]
Elapsed: 22 ms
Successful ops: 4000
Failed ops: 0
Total ops: 4000
Throughput: 181818 ops/sec
Average successful op latency: 21 us

[Write-heavy workload]
Elapsed: 5 ms
Successful ops: 4000
Failed ops: 0
Total ops: 4000
Throughput: 800000 ops/sec
Average successful op latency: 5 us

[Mixed multi-client workload]
Elapsed: 57 ms
Successful ops: 4000
Failed ops: 0
Total ops: 4000
Throughput: 70175 ops/sec
Average successful op latency: 56 us

Overall result: PASS
```

Figure 4: Benchmark result screenshot ss4.png.

```
ritvik@Ritviks-MacBook-Air FLexQL % ./build/benchmark_select
Preparing select benchmark tables...
Seed rows: 100000
Queries per set: 1000
Rounds: 5
Insert batch size: 1000

[Point SELECT benchmark]
Queries: 5000
Elapsed: 98 ms
Throughput: 51020 queries/sec
Average rows returned: 1

[Range SELECT benchmark]
Queries: 5000
Elapsed: 1177 ms
Throughput: 4248 queries/sec
Average rows returned: 200

[JOIN SELECT benchmark]
Queries: 5000
Elapsed: 83 ms
Throughput: 60240 queries/sec
Average rows returned: 1

[COUNT benchmark]
Queries: 5000
Elapsed: 731 ms
Throughput: 6839 queries/sec
Average rows returned: 1
```

Figure 5: Benchmark result screenshot ss5.png.

15 Conclusion

FlexQL satisfies the assignment constraints with a fully custom C++ implementation. It uses persistent disk-backed storage, supports fault-tolerant recovery, implements batch insertion,

supports the required SQL subset, includes an API-driven interactive REPL client, and includes additional measurement coverage beyond the provided benchmark.